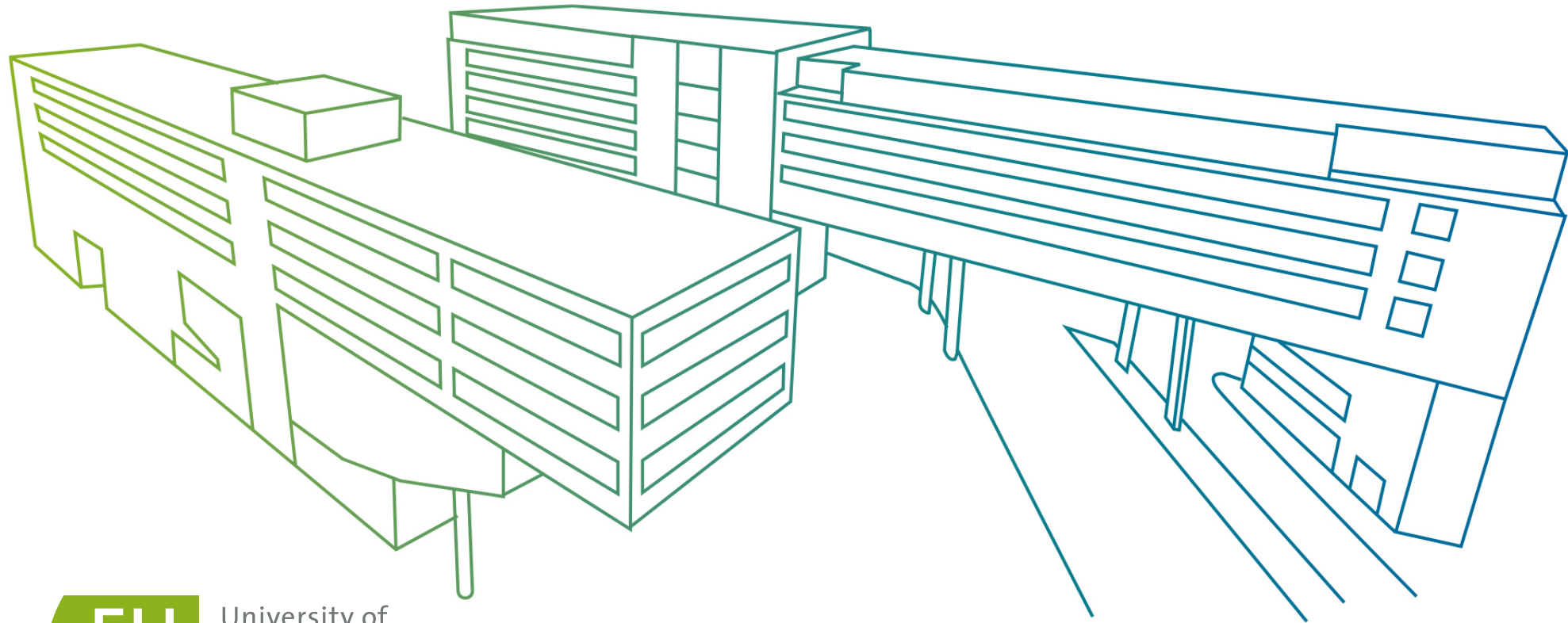




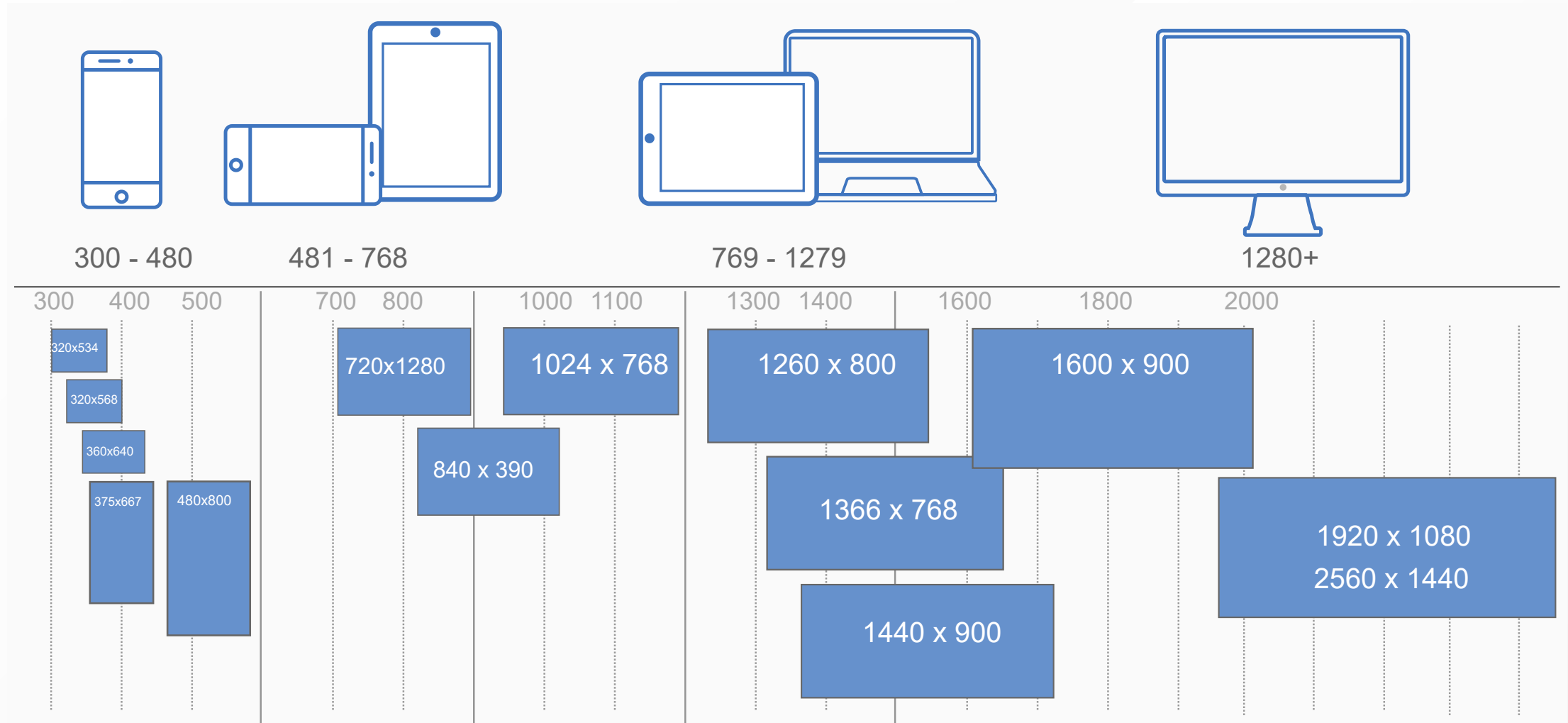
Web Technologies

CSS Advanced

Screen sizes

- Up until now we only designed our site for desktop users
- The navbar is long with many links
- Big cards are in rows besides each other
- How would that look like on a mobile device?
- What kind of other devices can you think of?

Screen sizes

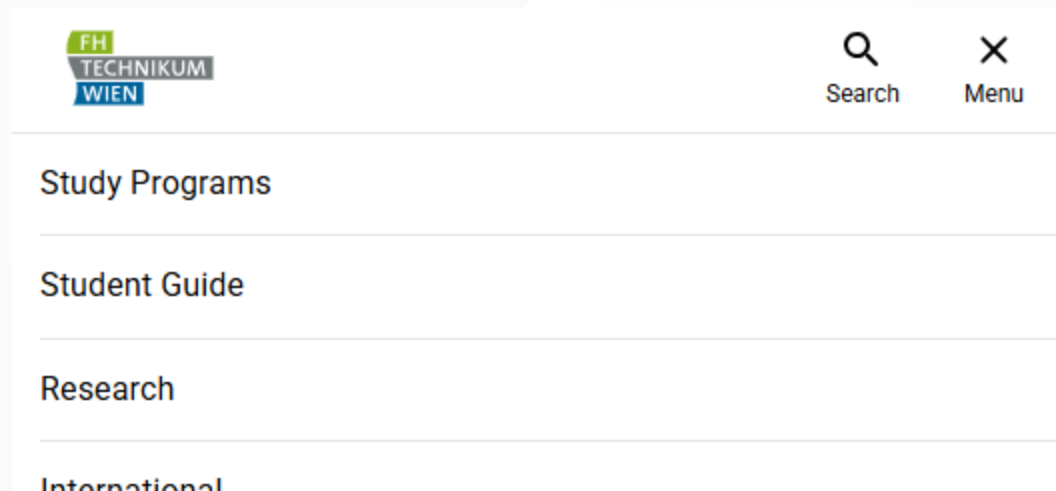
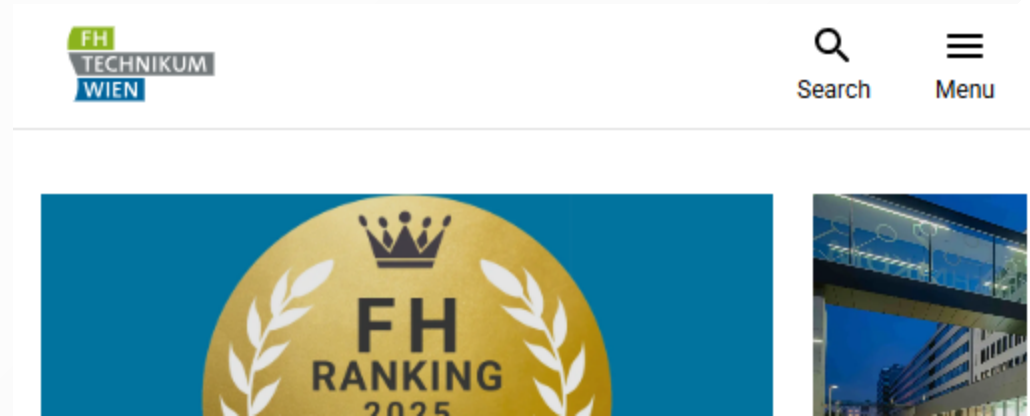


Questions

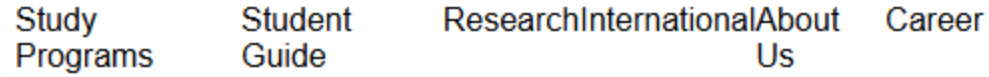
- Do we have to develop a new site for every screen?
 - No, luckily we can use *responsive* design
- Remember, HTML is all about content and structure
 - On desktop and mobile, the content and structure doesn't change
- We have to use CSS to change the layout for different screen sizes

But how...? First, let's look at the mobile mockup to gather requirements:

Mobile mockup



Our current "mobile" navbar



Study Programs Student Guide ResearchInternationalAbout Us Career

- As you can see, there is no menu option and all the links are just cramped together
- This is not mobile friendly
 - Or you could say, this is not *responsive*

Let's try to fix that!

Mobile requirements

- All links need to be under each other
- There needs to be a line between the links
- There is a menu option with three horizontal lines
 - This is called a burger menu
- The list of links is initially not visible
- If the menu option is clicked, the links are visible and the three lines "change" to a cross

Let's start with what we know!

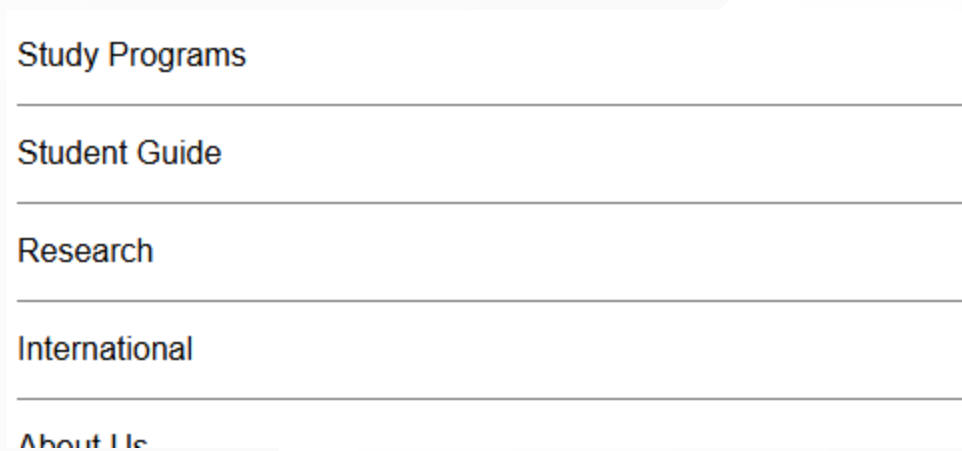
Flexbox to the rescue

At the beginning, we are going to ignore that the layout has to change for smaller screen, we just want to see if we can build the mockup. Here we can only see what we changed/added:

```
nav ul {  
    /* previous CSS */  
    flex-direction: column;  
}  
nav ul li a {  
    /* previous CSS */  
    padding: 15px 0;  
    border-bottom: 1px solid gray;  
}
```

flex-direction: column

- Again, Flexbox offers a simple way to arrange elements as columns one below the other



- In addition, we have added vertical padding (touch-friendly) and a border at the bottom as a dividing line

The breaking point

- Now we need to toggle the previous CSS for larger screens and the new CSS for smaller screens
- We need to define a breakpoint where we want to "switch"
 - Maybe somewhere between `600px` and `700px`?
 - This can change depending on modern trends
- CSS has a feature called: **Media queries**
- Media queries allow you to apply CSS styles depending on a device's media type
 - That includes screen size, resolution, orientation, ...

Media queries

A media query can look like this:

```
@media only screen and (max-width: 600px) {  
    /* your CSS */  
}
```

This would mean: *If the screen is 600 pixels wide or smaller (as is the case with most mobile phones), use the following CSS rules.*

So in theory, if we only add our new CSS from a couple slides ago into the media query, we can have both navbars!

Responsive design

Our old CSS (original navbar) is applied on desktop screens, but the new CSS (mobile view) is active on smaller screens.

```
@media only screen and (max-width: 600px) {  
  nav ul {  
    flex-direction: column;  
  }  
  nav ul li {  
    padding: 15px 0;  
    border-bottom: 1px solid gray;  
  }  
}
```

What first?

- What we did now is called "Desktop First Design"
 - We wrote CSS for large screens (with the desktop in mind)
 - We turn on extra rules for smaller screens (with a media query)
- You can also design "Mobile First"
 - Write CSS for small devices
 - And turn on extra rules for large screens
- For "Mobile First Design" you have to *reverse* the media query

```
@media only screen and (min-width: 600px) { /* only applies for 600+px */ }
```

The advanced part

- This was the easy part!
- Now we need to implement the menu option
- In the mobile view, the links should be hidden, until the menu option is selected
- There should even be an icon, that changes from three horizontal lines to a cross, if the links are visible
- How can we do this?

CSS can do a lot!

- If your first thought was JavaScript, yes you are not wrong
- But we actually don't *need* JavaScript for this
- We only know HTML and CSS by now, and believe it or not, that's enough for a mobile hamburger menu
- Don't believe me? I'll show you!
- But be cautious, we need everything we have learned so far!

Forms?

- Remember HTML forms?
- `<input>` and `type` and `<label>` and `for`
- But how can that possibly be useful for our hamburger menu?
- Essentially, we want to build a toggle
 - A toggle is a yes or no datatype, that can *toggle* elements
 - Our toggle needs to show and hide the links

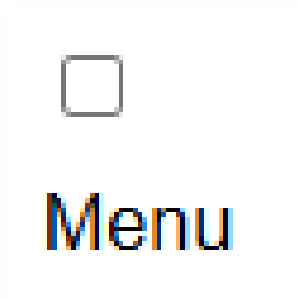
Can you think of an `<input>` `type` that can act as a toggle?

Toggles

- A checkbox, thats right!
- But we don't want to click on a checkbox, that's not in our design... we want to click on the word "Menu"
- There is another element that can toggle a checkbox, if it's setup correctly
- Depending on how much you explored HTML forms, you might have noticed, that clicking on a corresponding `<label>`, the input will be selected
 - Or in the case of a checkbox, toggled!

Add the checkbox to the navbar

```
<nav>
  <input id="menu-toggle" type="checkbox">
  <div>
    <label for="menu-toggle">
      Menu
    </label>
  </div>
  <!-- link list -->
</nav>
```



Labels are important

- We can try it out!
 - Try clicking on the word "Menu" and watch the checkbox toggle from being *unchecked* to *checked*
- "But we don't want a checkbox in the navbar" I hear you say
- Well, simply hide it!

```
#menu-toggle {  
  display: none;  
}
```

- Now we don't see it, but it's still there

How does that help us?

- Remember `:hover`, the CSS pseudo-class?
- It tracks the state of an element and only applies CSS rules, if the state is active

```
nav ul li a:hover {  
    text-decoration: underline;  
}
```

- Now wouldn't it be fantastic if we could track the state of a checkbox?... We can, it's called `:checked` !

Update design

- Let's add some CSS to put the menu option on the right side and add a border

```
#menu-toggle {  
    display: none; /* we don't want to see the checkbox */  
}  
#menu-toggle + div { /* what's with the "+"? */  
    display: none; /* remember "Desktop First?" */  
    justify-content: flex-end; /* puts "Menu" to the right */  
    padding: 10px 0;  
    border-bottom: 1px solid gray;  
}
```

Update media query

```
@media only screen and (max-width: 600px) {  
  nav ul {  
    margin: 0;  
    display: none; /* links are be hidden by default on small screens */  
    flex-direction: column;  
  }  
  nav ul li {  
    padding: 15px 0;  
    border-bottom: 1px solid gray;  
  }  
  #menu-toggle + div {  
    display: flex; /* only visible on small screens */  
  }  
}
```

Now the best part!

We add this to our media query:

```
@media only screen and (max-width: 600px) {  
    /* previous CSS */  
    #menu-toggle:checked ~ ul {  
        display: flex;  
    }  
}
```

Translation: *When the checkbox with the ID `menu-toggle` is `checked`, find the `<ul>` that comes after it, and make it display as a flexbox.*

What was previously hidden, is visible if the checkbox is checked!

Programming with CSS

- What we basically did was to program a simple *if statement* in CSS
- The `<input type="checkbox">` and pseudo-class `:checked` act as a variable, and the `<label>` acts as a function to *toggle* the value!
 - Advanced stuff!



Are we done?

- Not quite! There is still one requirement open
- We want three horizontal lines if the menu is closed and a cross if the menu is open
- There are Unicode characters we could use:  and 
- But how to add them to the HTML, if we cannot change it dynamically?

CSS again!

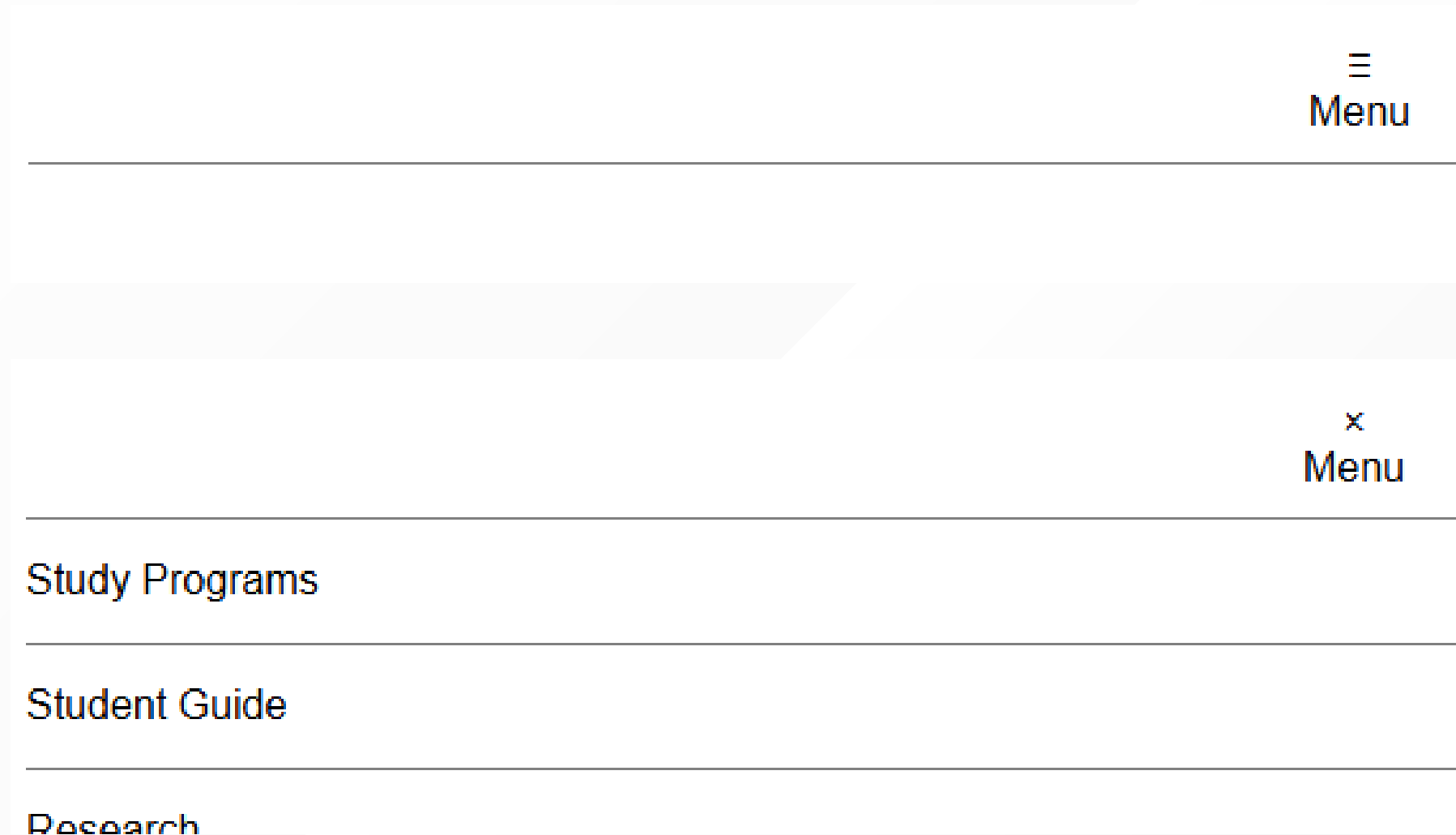
- We can add content to the CSS via pseudo-classes!
- Remember `::marker` from the original list?
- The content of that `::marker` was a `•`
- We can use that system and define our own content for the `::before` pseudo class
- And depending on the `:checked` status, we can change the content

::before

```
#menu-toggle + div label::before {  
    content: "≡";  
}  
#menu-toggle:checked + div label::before {  
    content: "×";  
}
```

- `#menu-toggle` selects the checkbox
- `+ div` says: Find the `<div>` that comes immediately after that checkbox.
- `label::before` means: Style the `<label>` inside that `<div>` by inserting content before it

The final result



The final result code

- You can find the full code for this example in the Moodle course
- It includes some additional CSS that was not discussed in the slides
 - Mainly Flexbox and CSS padding/margin to make the menu option a little more appealing
- If you couldn't follow every part of this class, that's okay, it was an advanced topic
- Try reading through the slides again from the beginning, try each step yourself, you will get there!

Key Takeaways

Responsive Design Fundamentals: Use one codebase for multiple screens. The content and structure stays the same, only the layout changes with CSS. Different user interaction patterns require different approaches (regarding spacing and presentation).

Media Queries: Choose strategic breakpoints for layout changes. Go with a "Mobile First" or "Desktop First" approach depending on your expected user base.

Key Takeaways

Advanced CSS: You can use hidden checkboxes and pseudo-classes for state management. No JavaScript needed!. CSS selectors like `+` and `~` let you select direct or adjacent sibling elements. Pseudo-elements like `::before` can add state-based content, without the need to modify the HTML. And CSS can do so much more, it's really impressive!

“ Start simple, understand each concept, then combine them to create powerful responsive experiences! ”